DUOC UC · ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe técnico

Desarrollo Backend Avanzado: Spring Cloud y Batch

Modernización del sistema legacy del Banco XYZ hacia microservicios en la nube

Asignatura: Desarrollo Backend III (PBY2203)

Evaluación: Evaluación Final Transversal · Semana 9

Estudiante: Bayro Rañileo

Docente: Alonso Esteban Castillo Pizarro

Fecha: Octubre de 2026

Repositorio: https://github.com/BayroBastian2025/EFT_S9_Bayro_Ranileo

1. Resumen ejecutivo

El Banco XYZ opera con un sistema legacy (COBOL y scripts Shell sobre mainframe) que procesa transacciones, calcula intereses, genera estados de cuenta y gestiona clientes. Ese sistema es difícil de escalar, caro de mantener y complicado de integrar con tecnologías nuevas.

Este proyecto lo moderniza con una arquitectura de microservicios en Spring Boot y Spring Cloud, empaquetada en Docker y lista para AWS. Se construyó sobre el trabajo de la semana anterior (OAuth2, BFF, Resilience4j, Kafka y Docker) y se le agregó lo que faltaba: los procesos batch con Spring Batch, la división del backend en tres microservicios de negocio, el registro de servicios (Eureka), la configuración centralizada (Config Server) y una puerta de entrada con balanceo de carga (API Gateway).

Resultado: 14 contenedores que se levantan con un solo comando (docker compose up), 3 procesos batch que procesan 1.000 registros por archivo omitiendo y documentando los datos inválidos, 3 BFF por canal, 3 microservicios de negocio resilientes, mensajería asíncrona y pruebas automáticas en Postman.

2. Procesos clave de la migración

Se identificaron cinco procesos críticos para pasar del sistema legacy a la arquitectura de microservicios:

#	Proceso clave	Problema del legacy	Qué se implementó
1	Migración de procesos batch a Spring Batch	Scripts sin reintentos, sin trazabilidad de errores y de ejecución secuencial	batch-service con 3 jobs: lectura, validación, escritura, skip, retry, multihilo y re-ejecución automática
2	División del monolito en microservicios	Un fallo en un módulo afecta a todo; difícil de escalar	Clientes, Cuentas y Pagos como servicios independientes con su propia base de datos
3	Patrón Backend for Frontend (BFF)	Todos los canales reciben los mismos datos	BFF Web (completo), BFF Móvil (liviano) y BFF Cajero (operaciones críticas)
4	Seguridad distribuida con Spring Security	Seguridad limitada y centralizada	OAuth2: Authorization Server con JWT y scopes por canal; cada servicio valida el token
5	Mensajería asíncrona con Kafka	Sistemas acoplados	Pagos publica eventos de transacciones; Notificaciones los consume y genera alertas

3. Propuesta de arquitectura y requerimientos clave del negocio

A partir del contexto del banco se definieron tres requerimientos clave del negocio. Cada decisión de diseño se justifica contra alguno de ellos.

Requerimiento del negocio	Decisión de arquitectura	Justificación
R1. Escalabilidad y bajo costo de operación (el legacy solo escala en vertical y es caro)	Microservicios en contenedores + Eureka + API Gateway con balanceo (lb://) + Docker Compose / AWS	Cada servicio se replica por separado según su demanda (ej. docker compose up --scale clientes-service=3). Eureka detecta las instancias nuevas sin cambiar configuración.
R2. Disponibilidad y continuidad de operaciones críticas (retiros, pagos)	Resilience4j (Circuit Breaker, Retry, Bulkhead, timeouts, fallback), Kafka y compensación (saga) entre Pagos y Cuentas	Si Cuentas cae, Pagos responde rápido con un 503 controlado en vez de colgarse. Un fallo de Kafka no revierte el retiro. Si algo falla a mitad de una transferencia, el dinero se devuelve.
R3. Seguridad de la información financiera y control por canal	OAuth2 con JWT firmado (RS256) y scopes por canal; secretos en variables de entorno; contenedores sin usuario root	Un token de cajero no sirve en la web ni en el móvil (principio de mínimo privilegio). Cada servicio valida el token por sí mismo: no hay un único punto de confianza.

Decisiones complementarias:

- **Configuración centralizada:** los parámetros de negocio (tasas de interés, máximo por transferencia, hilos del batch) están en el Config Server y se cambian sin recompilar.
- **Base de datos por servicio:** cada microservicio tiene su propia base de datos (H2 en este entorno de evaluación), de modo que pueda evolucionar y escalar de forma independiente.
- **Cambio mínimo en los BFF:** los BFF no cambiaron respecto a la semana anterior: solo apuntan al API Gateway. Esto permite que los equipos de frontend y backend avancen en paralelo.

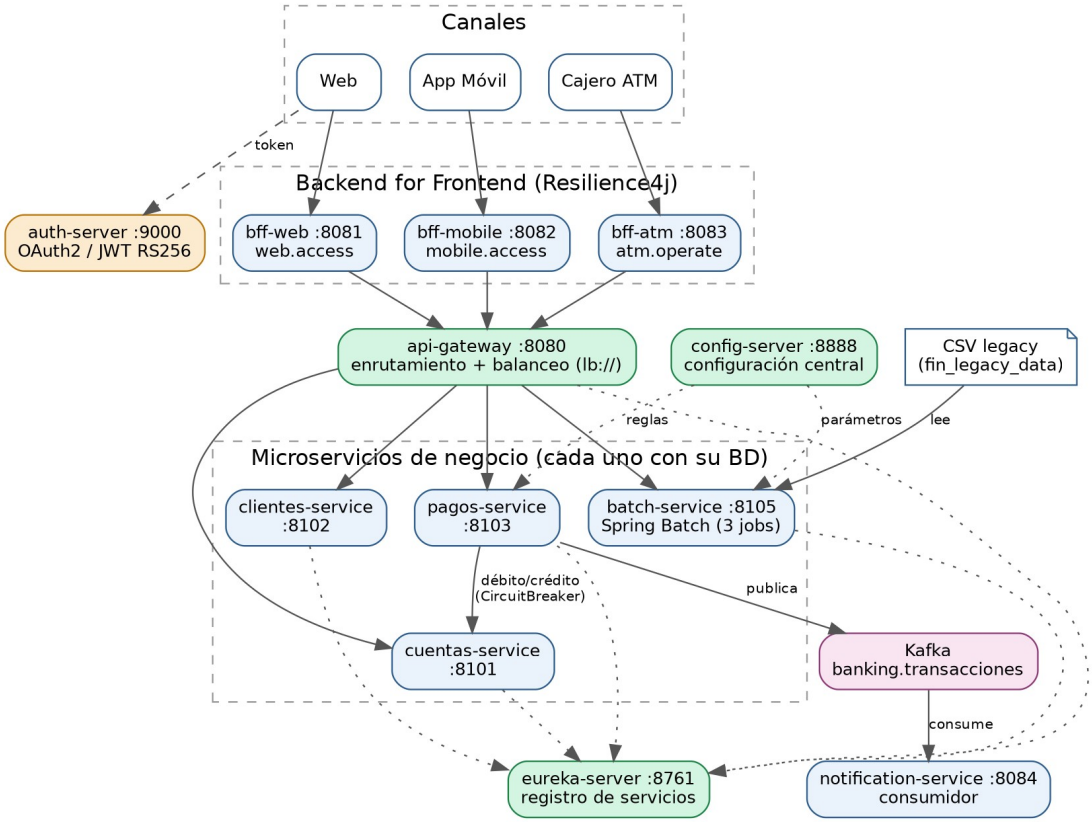


Figura 1. Diagrama de arquitectura de la plataforma.

3.1 Componentes

Componente	Puerto	Responsabilidad
auth-server	9000	Emite tokens OAuth2 (JWT). Publica las llaves públicas (JWKS).
eureka-server	8761	Registro y descubrimiento de servicios.
config-server	8888	Entrega configuración centralizada (carpeta config-repo).
api-gateway	8080	Puerta de entrada: enruta por ruta y balancea entre instancias.
clientes-service	8102	Alta, consulta y actualización de datos de clientes.
cuentas-service	8101	Apertura, cierre/bloqueo, consulta y movimiento de saldos.
pagos-service	8103	Retiros, depósitos, pagos y transferencias; historial; productor Kafka.
batch-service	8105	3 procesos batch con Spring Batch.
bff-web / bff-mobile / bff-atm	8081 / 8082 / 8083	Un backend por canal, con su propio

		scope.
notification-service	8084	Consume eventos de Kafka y genera notificaciones y alertas.
kafka / kafka-ui	29092 / 8090	Bus de eventos y su interfaz web.

3.2 Casos de uso

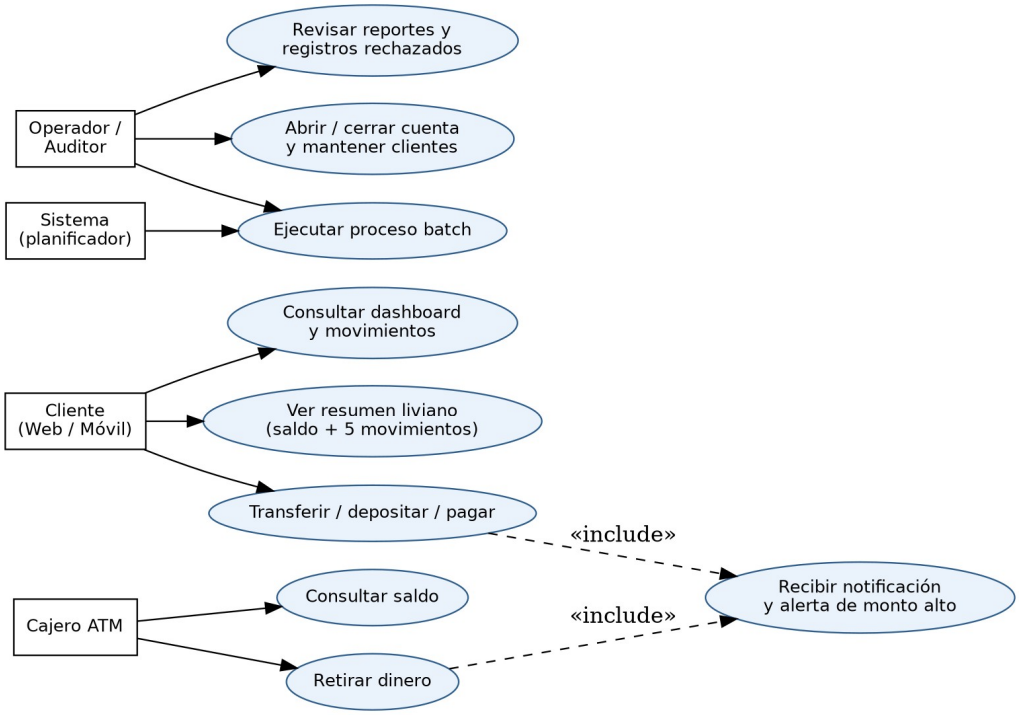


Figura 2. Casos de uso principales.

4. Migración de procesos batch (Spring Batch)

Los tres procesos del legacy se reescribieron como jobs de Spring Batch en el servicio batch-service. Los datos de entrada son los CSV del repositorio fin_legacy_data, que traen errores intencionales (fechas en 4 formatos, montos vacíos o negativos, tipos inválidos, edades imposibles y duplicados).

Job	Archivo de entrada	Reglas aplicadas	Salida
reporteTransaccionesDiariasJob	movimientos_financieros_diarios.csv	Fecha válida (4 formatos), monto mayor a 0, tipo crédito/débito. Marca como anomalía los montos altos (≥ $3.000).	Tabla y CSV con cantidad, total y anomalías por día y tipo
calculoInteresesTrimestralesJob	intereses_trimestrales.csv	Saldo presente y positivo, edad entre 18 y 120, tipo ahorro/préstamo/hipoteca, sin duplicados. Interés = saldo × tasa (ahorro se	Tabla y CSV con saldo e interés por tipo

		abona; préstamo e hipoteca se cargan).	
estadosFinancierosAnualesJob	estados_financieros_anuales.csv	Fecha válida, tipo normalizado (Depósito → deposito), monto distinto de cero, depósitos no negativos; descripción vacía se completa.	Tabla y CSV con ingresos, egresos y saldo neto por cuenta

4.1 Cómo se cumplen los requerimientos

- **Steps leer, procesar, escribir:** cada job tiene un paso de carga (lector CSV → procesador → escritor JDBC) y un paso final que genera el reporte.
- **Excepciones y reintentos:** las filas inválidas se omiten (skip) y se guardan con su motivo en la tabla registros_rechazados, sin detener el proceso. Los errores transitorios de base de datos se reintentan 3 veces con espera.
- **Optimización y volumen:** el paso de carga usa 4 hilos y bloques (chunks) de 50 registros, con un lector sincronizado y escritura por lotes.
- **Políticas de finalización y re-ejecución:** si los rechazos superan el límite (900), el job termina en FAILED. El orquestador lo vuelve a lanzar automáticamente hasta 3 veces con los mismos parámetros (Spring Batch lo trata como un reinicio).
- **Integridad y equivalencia:** la carga borra lo anterior antes de cada (re)ejecución, así un reintento no duplica registros. Las reglas replican las del legacy sobre los mismos archivos.

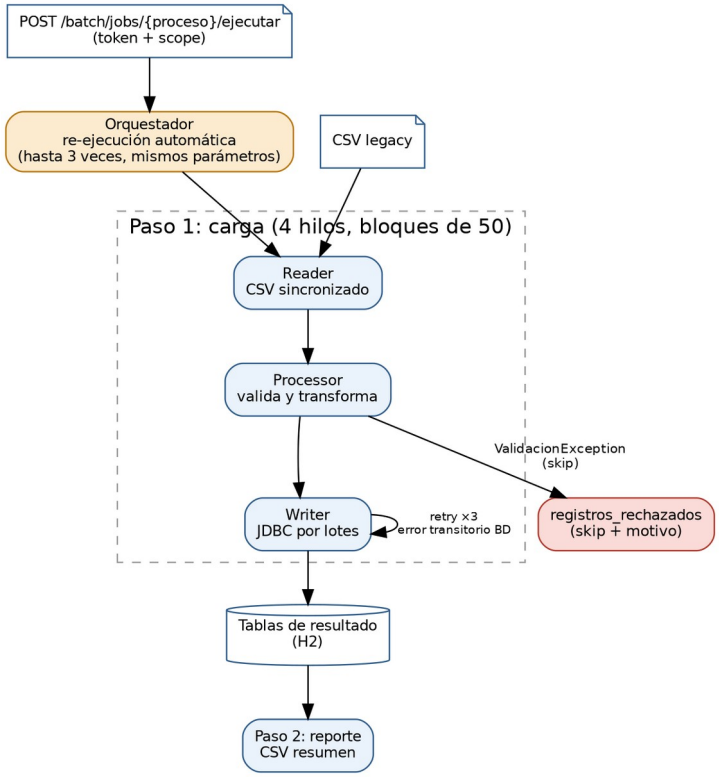


Figura 3. Flujo de un job de Spring Batch.

Resultados esperados con el archivo de 1.000 filas (semana_3):

Job	Leídas	Escritas (válidas)	Omitidas (rechazadas)
Transacciones diarias	1.000	392	608
Intereses trimestrales	1.000	421	579
Estados anuales	1.000	759	241

5. Patrón Backend for Frontend (BFF)

BFF	Scope	Optimización	Seguridad específica
bff-web (8081)	web.access	Dashboard completo: cliente, cuentas e historial completo	Historial degrada a lista vacía si el servicio falla (fallback)
bff-mobile (8082)	mobile.access	Respuesta liviana: saldo y solo los últimos 5 movimientos	Scope exclusivo de la app móvil
bff-atm (8083)	atm.operate	Solo saldo y retiro; valida el límite diario del cajero	El retiro no se reintenta (no es idempotente: evita descontar dos veces)

Cada BFF es independiente: tiene su propio código, configuración, scope y circuito de resiliencia. Para llamar al backend obtienen su propio token (client_credentials) y se comunican por el API Gateway. Las comunicaciones usan tokens firmados; en AWS se recomienda terminar HTTPS en un Application Load Balancer con certificado ACM (ver despliegue.md).

6. Microservicios resilientes con Spring Cloud

6.1 Los tres servicios clave

- **Gestión de Cuentas:** apertura de cuentas, cambio de estado (activa, bloqueada, cerrada; solo se cierra con saldo cero), consulta y movimientos de saldo (débito/crédito).
- **Procesamiento de Pagos:** retiros, depósitos, pagos y transferencias. Registra cada transacción, publica el evento en Kafka y coordina con Cuentas.
- **Gestión de Clientes:** alta de clientes, consulta y actualización de datos de contacto.

6.2 Consistencia en un entorno distribuido

Como Pagos y Cuentas tienen bases de datos separadas, no existe una transacción única. Se aplicó una compensación (saga simple): si algo falla después de mover el saldo, Pagos revierte el movimiento en Cuentas. Por ejemplo, en una transferencia a una cuenta inexistente el dinero vuelve a la cuenta de origen.

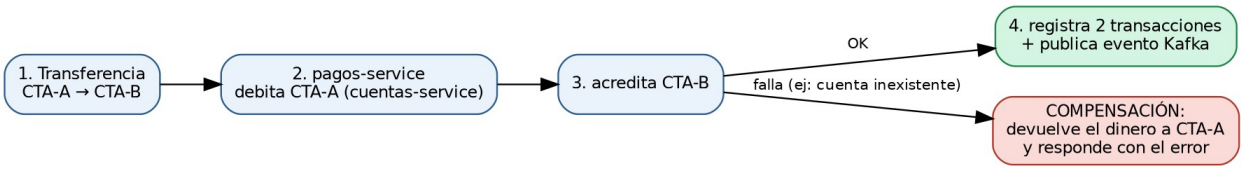


Figura 4. Flujo de una transferencia con compensación.

6.3 Resiliencia

Mecanismo	Configuración	Efecto
Timeouts	Conexión 2 s, lectura 3 s	Detecta caídas rápido
Retry	3 intentos con espera creciente (solo lecturas)	Absorbe fallos momentáneos
Circuit Breaker	Ventana de 10 llamadas; abre con 50% de fallos; 15–20 s abierto	Falla rápido sin golpear al servicio caído; se recupera solo
Bulkhead	10 llamadas simultáneas (5 en escritura)	Evita saturar el servicio
Fallback	Respuesta 503 con mensaje claro; los errores 404/409/422 se propagan como respuestas de negocio	Comportamiento alternativo controlado

6.4 Mensajería asíncrona (Kafka)

Pagos publica un evento TransaccionEvent en el tópico banking.transacciones (3 particiones; la clave es el número de cuenta para mantener el orden por cuenta) cada vez que registra una transacción. notification-service lo consume y genera una notificación INFO, o ALERTA_MONTO_ALTO si el monto es de $200.000 o más. Si Kafka no está disponible, la operación bancaria no falla: el error se registra y se continúa.

6.5 Seguridad distribuida

El auth-server emite tokens JWT firmados con RS256. Todos los servicios son Resource Servers: validan la firma con las llaves públicas (JWKS) y exigen el scope correspondiente (banking.read para consultas, banking.write para operaciones). Pagos reenvía el token recibido a Cuentas (token relay). Sin token la respuesta es 401; con un scope incorrecto, 403.

6.6 Configuración, descubrimiento y balanceo

- **Spring Cloud Config:** el Config Server (modo native) entrega parámetros a Pagos (máximo por transferencia) y a Batch (tasas, hilos, límites).
- **Eureka:** todos los servicios de negocio se registran y se encuentran por nombre (por ejemplo, Pagos llama a http://cuentas-service).
- **API Gateway:** el API Gateway enruta por ruta (/api/legacy/clientes → Clientes, /api/legacy/cuentas/*/retiro → Pagos, /batch → Batch, etc.) y reparte la carga entre las instancias registradas.

7. Despliegue y escalabilidad

- **Docker:** cada servicio tiene un Dockerfile multi-etapa (compila con Maven y ejecuta en una imagen liviana de Java 17 con usuario sin privilegios y HEALTHCHECK).
- **Docker Compose:** un solo archivo levanta los 14 contenedores en el orden correcto (esperando que las dependencias estén sanas) y con reinicio automático.
- **AWS:** se despliega en una instancia EC2 con el mismo docker-compose.yml (pasos en despliegue.md); a futuro: ECS Fargate, MSK, RDS y Secrets Manager.
- **Escalabilidad horizontal:** docker compose up -d --scale clientes-service=2 crea una segunda instancia que Eureka registra y el gateway utiliza.
- **Monitoreo:** Actuator (health, circuitbreakers, retries, bulkheads), logs de cada job con leídos/escritos/omitidos, logs de eventos Kafka, Kafka UI y el panel de Eureka.

8. Comparación con el sistema legacy

Aspecto	Legacy (COBOL + Shell)	Nuevo sistema
Procesos batch	Scripts sin reintentos ni registro de rechazos	Jobs con skip/retry, rechazos con motivo, reinicio automático y 4 hilos
Arquitectura	Monolito; un fallo afecta a todo	Microservicios con Circuit Breaker
Escalabilidad	Vertical (mainframe)	Horizontal (más instancias)
Seguridad	Centralizada y limitada	OAuth2 distribuido con scopes por canal
Integración	Acoplada	Eventos asíncronos con Kafka
Configuración	Dentro de scripts y código	Centralizada en Config Server
Despliegue	Manual en servidores	Un comando con Docker Compose; AWS

9. Desafíos enfrentados y soluciones

Desafío	Solución
Dividir el core sin romper los BFF ni Postman	Se mantuvieron las mismas URLs: el API Gateway enruta cada ruta al microservicio correcto y toma el puerto 8080 que usaba el core.
Datos legacy con errores (fechas en 4 formatos, vacíos, duplicados)	Procesadores con validaciones por regla; las filas inválidas se omiten con skip y se guardan con su motivo.
Reintentos sin duplicar datos	La carga se limpia antes de cada ejecución; los reintentos usan los mismos parámetros (reinicio de Spring Batch).
Detección de duplicados con varios hilos	Conjunto seguro para hilos que se crea nuevo en cada ejecución del paso.
Consistencia entre bases de datos separadas	Compensación (saga simple) entre Pagos y Cuentas.
Retiro no idempotente	El retiro y los movimientos de saldo no llevan Retry, solo

	Circuit Breaker y Bulkhead.
Evitar que la configuración central pisara ajustes locales	El config-repo no define endpoints de Actuator ni usa variables ${}.
Consumo de memoria con 14 contenedores	JVM acotada a 384 MB por servicio y recomendación de instancia t3.xlarge en AWS.

10. Propuestas de mejora y próximos pasos

- Reemplazar H2 por PostgreSQL/RDS (una base por servicio) para persistir los datos y poder escalar Cuentas y Pagos.
- HTTPS de extremo a extremo (Application Load Balancer con certificado ACM) y secretos en AWS Secrets Manager.
- Kafka administrado (Amazon MSK) con TLS y un tópico dedicado a alertas de seguridad; dead-letter topic en el consumidor.
- Rate limiting en el API Gateway y trazabilidad distribuida (Micrometer + Zipkin).
- Entrada y salida de los batch en Amazon S3 y ejecución programada (por ejemplo, EventBridge).
- Pipeline CI/CD (GitHub Actions → ECR → ECS) y repositorio Git para el Config Server.

11. Evidencias de ejecución

Capturas de la ejecución del sistema (ver guía en evidencia-ejecucion/README.md).

Figura 5. docker compose ps con todos los servicios healthy.

[Pegar aquí la captura: docker compose ps con todos los servicios healthy]

Figura 6. Panel de Eureka con los servicios registrados.

[Pegar aquí la captura: Panel de Eureka con los servicios registrados]

Figura 7. Config Server entregando la configuración de batch-service.

[Pegar aquí la captura: Config Server entregando la configuración de batch-service]

Figura 8. Postman: 3 jobs COMPLETED (leídos / escritos / omitidos).

[Pegar aquí la captura: Postman: 3 jobs COMPLETED (leídos / escritos / omitidos)]

Figura 9. Reporte de intereses y registros rechazados con su motivo.

[Pegar aquí la captura: Reporte de intereses y registros rechazados con su motivo]

Figura 10. Archivos CSV generados en /app/output.

[Pegar aquí la captura: Archivos CSV generados en /app/output]

Figura 11. Postman: depósito, transferencia y transferencia con compensación (404).

[Pegar aquí la captura: Postman: depósito, transferencia y transferencia con compensación (404)]

Figura 12. Kafka UI: eventos en banking.transacciones y notificaciones.

[Pegar aquí la captura: Kafka UI: eventos en banking.transacciones y notificaciones]

Figura 13. Circuit Breaker abierto (503) y recuperación (CLOSED).

[Pegar aquí la captura: Circuit Breaker abierto (503) y recuperación (CLOSED)]

Figura 14. Escalado: CLIENTES-SERVICE con 2 instancias en Eureka.

[Pegar aquí la captura: Escalado: CLIENTES-SERVICE con 2 instancias en Eureka]

Anexo. Relación con la pauta de evaluación

N°	Criterio	Dónde se cubre
1	Procesos clave identificados	Sección 2
2	Propuesta de arquitectura y requerimientos del negocio	Sección 3
3	Procesos batch (errores, paralelismo, escalabilidad)	Sección 4
4	BFF para 3 canales	Sección 5
5	Microservicios resilientes con Resilience4j y Kafka	Sección 6
6	Docker y escalabilidad horizontal	Sección 7
7	Documentación: código, informe,	readme.md, este informe,

	instrucciones y despliegue	instrucciones.md, despliegue.md
8	Video	Presentación de 5 a 7 minutos